

Using the UNITY Cluster to Accelerate Molecular Docking

By Aivy Le

Draft

BIOL. 4062 – Dr. Chain

March 30th, 2026

Presentation: [UNITYpresentation.mp4](#)

Video Tutorial: [UNITYtutorial.mp4](#)

GitHub: <https://github.com/aivy00/UNITYmoleculardocking>

Table of Contents

Background Information	3
Purpose	4
Research Question.....	4
Software Tools, Databases, & Datasets	5
Prerequisites & Set Up	5
Tutorial - Using UNITY Batch Jobs for Molecular Docking	7
Conclusion & Data Interpretation	15
References	16

Background Information

Molecular docking is a computational technique that is used to predict how a small molecule (ligand) fits into and binds with a target such as a receptor protein. It is commonly used in drug discovery because it allows researchers to screen large libraries of potential compounds and identify those most likely to bind effectively to a target. By evaluating possible in-silico interactions, molecular docking helps narrow down the extensive number of candidate drugs before proceeding to experimental validation (Agu et al., 2023). This significantly reduces both time and cost in the drug discovery process.

One widely used tool for molecular docking is *AutoDock Vina* (<https://vina.scripps.edu/>) which estimates binding affinity and provides a predicted 3-D model of the ligand-protein interaction. This allows researchers to assess the strength of the binding and the structural orientation of the ligand within the binding site (Agu et al., 2023).

Today, advancements in computational technologies have transformed molecular docking. High performance computing (HPC) systems have become a powerful tool for scaling molecular docking workflows to handle large datasets. An HPC system is made up of many interconnected computers, called computing nodes, that work together to complete tasks. Instead of running a single docking job on one computer, the workload can be divided into smaller tasks and run at the same time across multiple nodes. This allows researchers to dock thousands of ligands simultaneously rather than one at a time (Puertas-Martín et al., 2020). Additionally, these systems typically operate in Unix environments and rely on bash scripting to manage workflows. Bash scripting is a way of writing scripts that tell a computer running in the Unix system what to do instead of writing commands one by one. Overall, HPC systems not only save time but also make it possible to analyze large datasets. For example, studies that would normally take days or weeks can be completed in only a couple of hours.

The HPC platform demonstrated in this tutorial is *UNITY* (<https://unityhpc.org/>) which is a collaborative, multi-institutional resource shared amongst researchers across several universities on the east coast.

Purpose

This manual serves as a guide for researchers to use the UNITY high-performance computing platform to perform large-scale molecular docking with AutoDock Vina.

It will demonstrate how to prepare and execute multiple ligand docking jobs across multiple nodes to enable faster molecular docking, large-scale screening for drug discovery, and molecular modeling projects.

Research Question

ERAP1, ERAP2, and IRAP are proteins that belong to the M1 aminopeptidase family. They are a group of metalloenzymes that play an important role in peptide trimming for antigen presentation (Barlow & Thompson, 2020). Dysregulation of these enzymes have been linked to several autoimmune diseases such as ankylosing spondylitis, Behçet's disease, and type I diabetes (Barlow & Thompson, 2020). Unfortunately, efforts to develop drugs to treat these diseases have been challenging due to the high-similarity of their N-terminal catalytic sites. Drugs that are meant to bind to only one protein will bind to the others because of how similar their N-terminal sites are. To address this, recent research has identified a C-terminal regulatory region known as the SC site in ERAP1 (Sui & Guo, 2021). This SC site may serve as a more selective therapeutic target because it is different in ERAP1, ERAP2, and IRAP. By performing molecular docking in this region, there is potential for identifying ligands that can selectively modulate one protein over the others.

This manual will demonstrate how to dock the regulatory SC sites of the proteins ERAP1, ERAP2, and IRAP using large-scale computational ligand screening to identify a selective modulator that binds strongly to one protein. Specifically, the goal is to find a ligand that binds strongly to one of these proteins while showing weaker binding to the other two proteins. Because this process involves docking many ligands across three different proteins, an HPC platform is used to accelerate the workflow.

Software Tools, Databases, & Datasets

Software Tools:

1. **UNITY:** <https://unityhpc.org/>
 - a. Access to UNITY is *required*
 - b. Make sure to have an account
 - c. No installation is needed
2. **AutoDock Vina:** <https://vina.scripps.edu/downloads/>
 - a. This is the molecular docking tool we will be using
 - b. Install the Linux version: *autodock_vina_1_1_2_linux_x86.tgz*
3. **MobaXterm:** <https://mobaxterm.mobatek.net/>
 - a. A simple UI terminal we will run UNITY through
 - b. It allows us to easily upload & extract files from our PC
4. **Visual Studio Code (VS Code):** <https://code.visualstudio.com/>
 - a. Python scripts are needed to tell each computer what to do
 - b. VS Code is an editor where you can create and edit Python scripts
 - It contains an optional AI helper for beginner coders
 - The script used in this tutorial is uploaded to github for easy access
 - It tells the computer to run AutoDock Vina on every ligand file in the directory

Databases & Datasets:

1. **Protein Data Bank:** <https://www.rcsb.org/>
 - a. A database to obtain the receptor protein files
 - b. Download each receptor file as 'Legacy PDB Format'
 - c. In this guide, we will be using these three proteins:
 - i. [6T6R \(ERAP1\)](#)
 - ii. [6EA4 \(ERAP2\)](#)
 - iii. [5MJ6 \(IRAP\)](#)
2. **ZINC20:** <https://zinc.docking.org/>
 - a. A database to download groups of ligands in *.pdbqt* format
 - This format is needed for AutoDock Vina
 - b. In this guide, we will be docking ligands from the **EIAB tranche**
 - A tranche is a group of molecules in ZINC that contain similar properties
 - This tranche contains 23,273 ligands
 - **Reason why we are using this tranche:** Liddle et al. (2020) found a selective modulator for the SC site of ERAP1 called MNZ. This molecule is known as ZINC000035458855 in the ZINC database. MNZ falls into the EIAB tranche which contains other molecules that are in a similar weight range and properties to MNZ. By docking other molecules with the same properties as MNZ, another selective modulator may be found.
 - c. Optional database:
 - **PubChem:** <https://pubchem.ncbi.nlm.nih.gov/>

Prerequisites & Set Up

This tutorial assumes prior knowledge with AutoDock, including preparing input files for docking and generating grid parameter files. Users who are unfamiliar with these should review the following [tutorial](#).

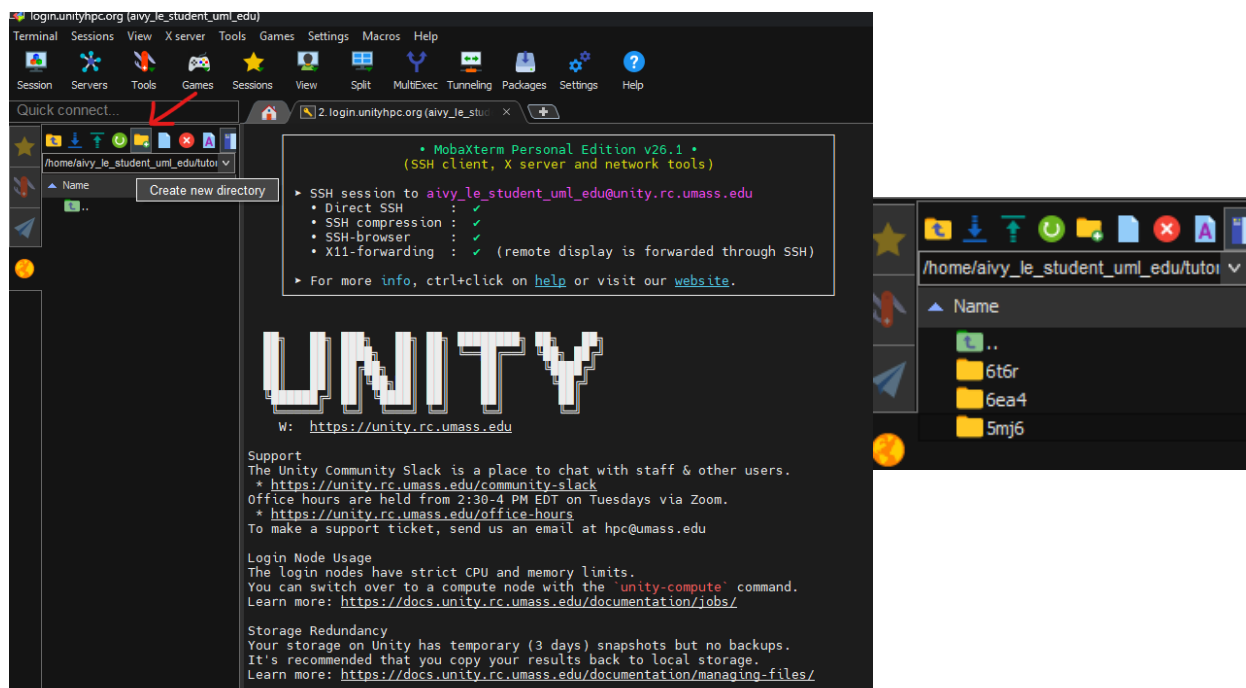
In addition, users should have remote access to the UNITY cluster through SSH. This includes generating an SSH key and successfully connecting through MobaXterm and VS Code. The following resources provide help for setting up:

1. [Obtaining SSH access for UNITY](#)
2. [Connecting via SSH using MobaXterm](#)
3. [Connecting via SSH using VS Code](#)

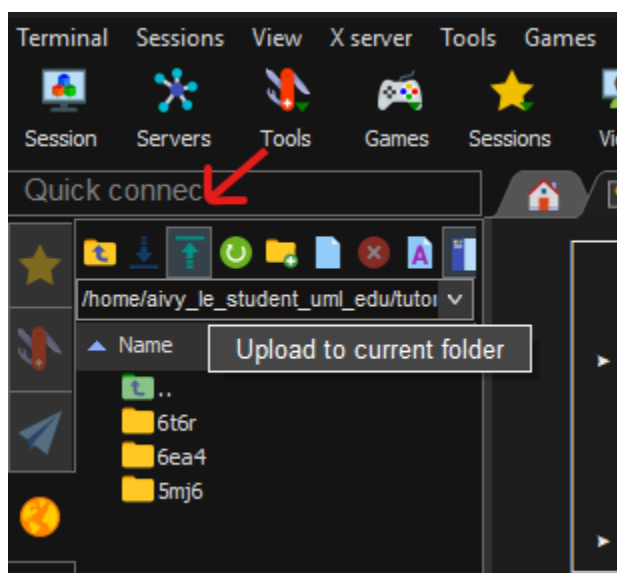
Files required	Obtained from	Information
6t6r.pdbqt (ERAP1)	Protein Data Bank	The prepared protein structure files for each protein being docked: ERAP1, ERAP2, and IRAP.
6ea4.pdbqt (ERAP2)	Protein Data Bank	
5mj6.pdbqt (IRAP)	Protein Data Bank	
conf_vs.txt (for 6t6r)	AutoDock Tools	Grid coordinate files for each receptor protein, indicating the location for ligand docking.
conf_vs.txt (for 6ea4)	AutoDock Tools	
conf_vs.txt (for 5mj6)	AutoDock Tools	
<i>All ligand files in .pdbqt format:</i> (example) EIABRN.xaa_ligand_15965.pdbqt	ZINC Database	Each file is a ligand molecule that will be docked into the receptor protein. There should be 23,273 ligands from the EIAB tranche.
vina.exe	AutoDock Vina (This will be downloaded in the UNITY shell on MobaXterm)	The tool that is used for molecular docking.
run_vina.py	VS Code (Uploaded on GitHub)	Python script that includes directions for docking with AutoDock Vina.

Tutorial - Using UNITY Batch Jobs for Molecular Docking

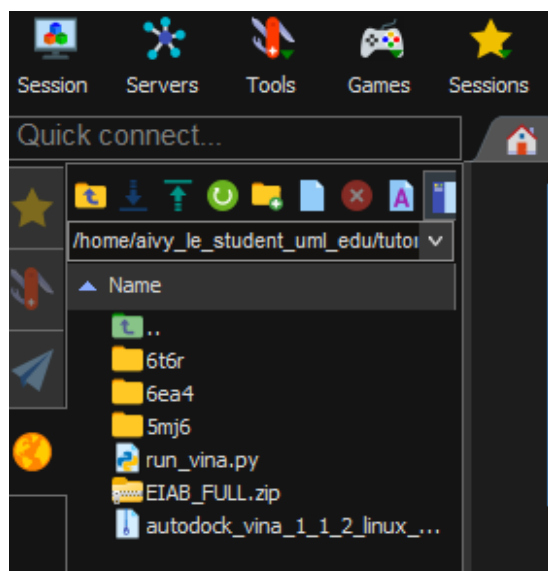
1. **Creating a new directory and uploading files:** Open up MobaXterm and create a folder. In this tutorial, a folder named 'tutorial' is created by clicking the icon and creating a new directory. Then, in this directory, create 3 more folders called: 6t6r, 6ea4, and 5mj6.



Next, click the *upload icon* and upload each file to the corresponding directory: (refer to the following sections for more information on each file – [Software Tools](#), [Databases, & Datasets](#) and [Pre-requisites & Set up](#))



Directory (folder)	File	Obtained from
tutorial	EIAB_FULL.zip	Contains all of the ligand files that were downloaded from the ZINC database .
	run_vina.py	Uploaded on GitHub
	autodock_vina_1_1_2_linux_x86.tgz	Linux version download from AutoDock website
6t6r	6t6r.pdbqt	Protein Data Bank: 6T6R
	conf_vs.txt (6t6r)	Created from AutoDock Tools - Uploaded to GitHub
6ea4	6ea4.pdbqt	Protein Data Bank: 6EA4
	conf_vs.txt (6ea4)	Created from AutoDock Tools - Uploaded to GitHub
5mj6	5mj6.pdbqt	Protein Data Bank: 5MJ6
	conf_vs.txt (5mj6)	Created from AutoDock Tools - Uploaded to GitHub



2. **Unzip the ligand folder (EIAB_FULL.zip) and the vina file:** Make sure you are in the correct directory first by typing `cd tutorial`. Then write the following code to unzip the ligand folder (EIAB_FULL.zip): `unzip EIAB_FULL.zip`

Adjust the input code based on your own file names (if they are different)

```
aivy_le_student_uhl_edu@login4:~$ cd tutorial
aivy_le_student_uhl_edu@login4:~/tutorial$ unzip EIAB_FULL.zip
Archive:  EIAB_FULL.zip
  creating: EIAB_FULL/
  inflating: EIAB_FULL/EIABMM.xaa.pdbqt
  inflating: EIAB_FULL/EIABMN.xaa.pdbqt
  inflating: EIAB_FULL/EIABMO.xaa.pdbqt
  inflating: EIAB_FULL/EIABMP.xaa.pdbqt
  inflating: EIAB_FULL/EIABRL.xaa.pdbqt
  inflating: EIAB_FULL/EIABRM.xaa.pdbqt
  inflating: EIAB_FULL/EIABRN.xaa.pdbqt
  inflating: EIAB_FULL/EIABRO.xaa.pdbqt
  inflating: EIAB_FULL/EIABRP.xaa.pdbqt
aivy_le_student_uhl_edu@login4:~/tutorial$
```

Next, the AutoDock Vina file must be unzipped following this code:

`tar -xvzf autodock_vina_1_1_2_linux_x86.tgz`

```
aivy_le_student_uhl_edu@login4:~/tutorial$ tar -xvzf autodock_vina_1_1_2_linux_x86.tgz
autodock_vina_1_1_2_linux_x86/
autodock_vina_1_1_2_linux_x86/LICENSE
autodock_vina_1_1_2_linux_x86/bin/
autodock_vina_1_1_2_linux_x86/bin/vina
autodock_vina_1_1_2_linux_x86/bin/vina_split
aivy_le_student_uhl_edu@login4:~/tutorial$
```

There should now be a folder containing *vina_split.exe* and *vina.exe*. Move the *vina.exe* file into the *tutorial* directory.

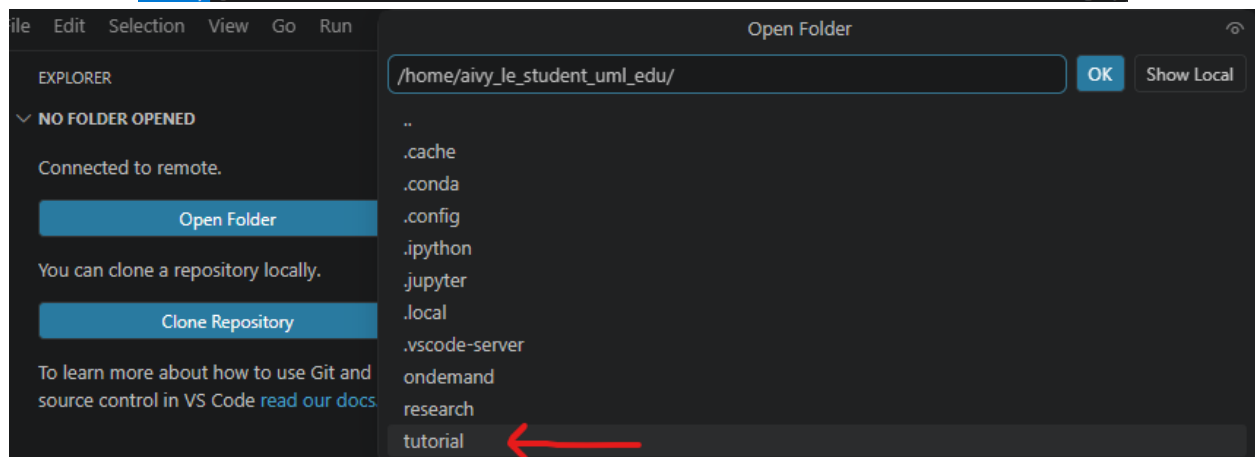
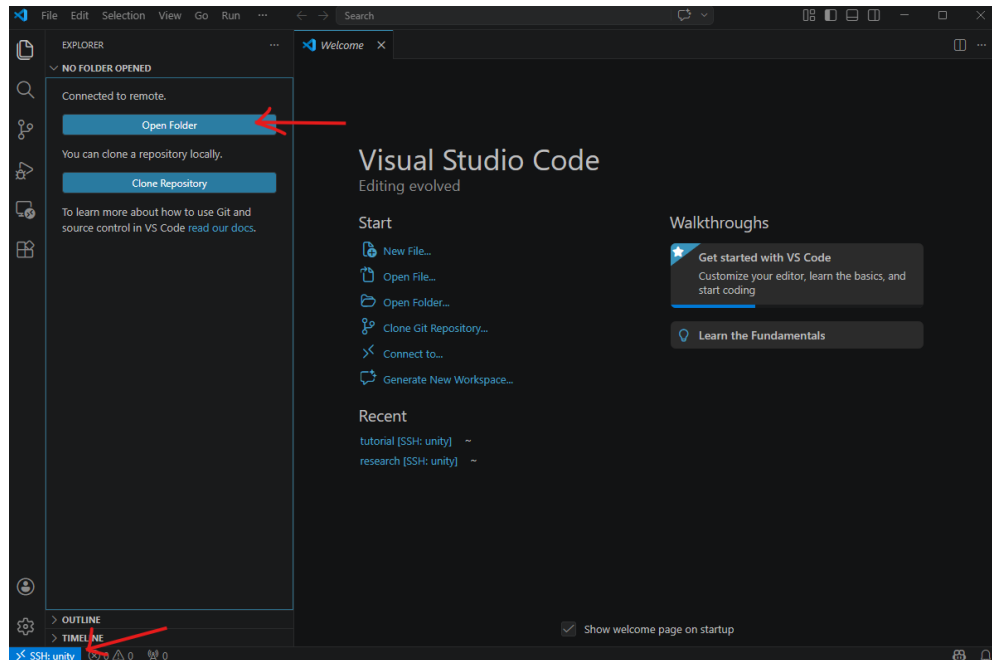
```
aivy_le_student_uhl_edu@login4:~/tutorial$ cd autodock_vina_1_1_2_linux_x86/bin
aivy_le_student_uhl_edu@login4:~/tutorial/autodock_vina_1_1_2_linux_x86/bin$ mv vina ../../
```

3. **Double-check that *run_vina.py* is in the *tutorial* directory:** Open VS Code to look at *run_vina.py*. This script is provided in [GitHub](#) for easy download and should already be uploaded in the *tutorial* directory. In summary, it will run through each ligand file in a directory and dock it with Vina. The output files will go into another directory that will be specified. Additionally, it will give a progress bar as it goes through each ligand. Here is an example of what that looks like:

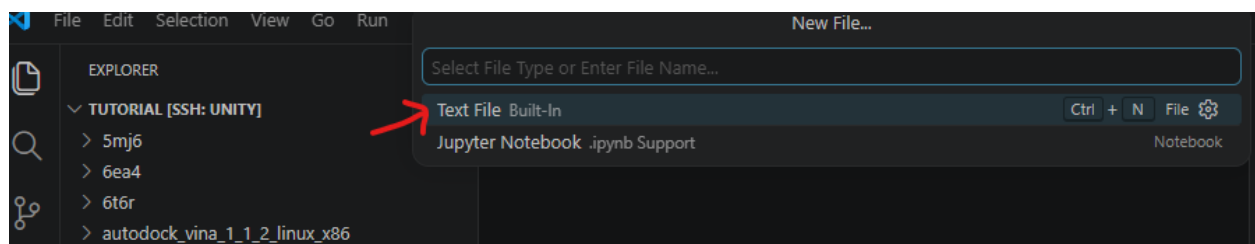
```
Ligands: 32% | 7498/23273 [2:45:40<9:42:26]#####
```

4. **Create a batch script:** A batch script will be used to queue a job in the UNITY cluster. All of the documentation on how to create a batch script can be found here: <https://docs.unity.rc.umass.edu/documentation/jobs/sbatch/#create-and-submit-a-batch-job>

First, open VS Code and make sure you are connected to the UNITY SSH. Then open the *tutorial* folder.



Now, create a new text file by clicking 'New File' → 'Text File'



Follow the batch script format shown on the [UNITY website](#) and add any parameters that are needed. Adjust the parameters based on what you want: number of cores used, memory per node, job time limit, and email options. Here is an example:

```
1  #!/bin/bash
2  #SBATCH -c 64 # Number of Cores per MPI Task
3  #SBATCH --mem=64G # Requested Memory Per Node
4  #SBATCH -p cpu # Partition
5  #SBATCH -t 08:00:00 # Job time limit
6  #SBATCH -o slurm-%j.out # %j = job ID
7  #SBATCH --mail-type=begin # send email when job begins
8  #SBATCH --mail-type=end # send email when job ends
9  #SBATCH --mail-user=aivy_le@student.uml.edu
10 #SBATCH --mail-type=TIME_LIMIT_80
11
```

Next, add the script. In this case, there will be a line of code to load our environment and then a line to run our run_vina.py script. The arguments for vina should be changed depending on the protein that is being docked in this batch job. In this example, 6t6r (ERAP1) is being docked so the location of each corresponding file must be directed.

--config: the configuration file containing the grid coordinates are in the 6t6r folder and is named conf_vs.txt

--receptor: the receptor protein is 6t6r.pdbqt, also located in the 6t6r folder

--ligand_dir: the ligand directory that contains all the ligands to be docked is the EIAB_FULL folder

--output_dir: the directory that all the output files should go to is the 6t6r folder

--vina_executable: indicates the vina file

--num_workers: indicates how many cores will be used

```
12 # load venv
13 source ./venv/bin/activate
14
15 # run the script with arguments for vina
16 python3 run_vina.py --config 6t6r/conf_vs.txt --receptor 6t6r/6t6r.pdbqt --ligand_dir EIAB_FULL/ --output_dir 6t6r/ --vina_executable vina --num_workers 32
```

Finally, save this file as **dock6t6r.sh**. Create 2 copies of this file and change the arguments to correspond to 6ea4 and 5mj6. Then name those files **dock6ea4.sh** and **dock5mj6.sh** accordingly. There should be 3 batch scripts.

The entire batch script should look like this:

```
$ dock6t6r.sh X
$ dock6t6r.sh
1  #!/bin/bash
2  #SBATCH -c 64 # Number of Cores per MPI Task
3  #SBATCH --mem=64G # Requested Memory Per Node
4  #SBATCH -p cpu # Partition
5  #SBATCH -t 08:00:00 # Job time limit
6  #SBATCH -o slurm-%j.out # %j = job ID
7  #SBATCH --mail-type=begin # send email when job begins
8  #SBATCH --mail-type=end # send email when job ends
9  #SBATCH --mail-user=aivy_le@student.uml.edu
10 #SBATCH --mail-type=TIME_LIMIT_80
11
12 # load venv
13 source ./venv/bin/activate
14
15 # run the script with arguments for vina
16 python3 run_vina.py --config 6t6r/conf_vs.txt --receptor 6t6r/6t6r.pdbqt --ligand_dir EIAB_FULL/ --output_dir 6t6r/ --vina_executable vina --num_workers 32
```

5. **Send batch jobs:** Go into MobaXterm, login to UNITY, and go to the tutorial directory. Then, use the command: `sbatch (batch script name)` to send in a batch job. There will be 3 jobs sent; one for each protein being docked.

```
aivy_le_student_uhl_edu@login1:~/tutorial$ sbatch dock6t6r.sh
sbatch: INFO: Note that 64 GB per node will require a node with more than 64 GB memory because of overhead. Check https://docs.unity.rc.umass.edu/nodes for an appropriate limit.
Submitted batch job 54484594
aivy_le_student_uhl_edu@login1:~/tutorial$ sbatch dock6ea4.sh
sbatch: INFO: Note that 64 GB per node will require a node with more than 64 GB memory because of overhead. Check https://docs.unity.rc.umass.edu/nodes for an appropriate limit.
Submitted batch job 54484595
aivy_le_student_uhl_edu@login1:~/tutorial$ sbatch dock5mj6.sh
sbatch: INFO: Note that 64 GB per node will require a node with more than 64 GB memory because of overhead. Check https://docs.unity.rc.umass.edu/nodes for an appropriate limit.
Submitted batch job 54484626
aivy_le_student_uhl_edu@login1:~/tutorial$
```

6. **Check the status of your batch jobs:** Use the command `squeue --me` to check what jobs you have queued and how long they have been running for. An output file with the job ID as the name will appear in the tutorial directory once the job has finished or the time has run out. The output file contains everything that has run during the job.

```
aivy_le_student_uhl_edu@login1:~/tutorial$ squeue --me
```

JOBID	PARTITION	NAME	USER	ST	TIME	NODES	NODELIST(Reason)
54484626	cpu	dock5mj6	aivy_le	PD	0:00	1	(Priority)
54484595	cpu	dock6ea4	aivy_le	PD	0:00	1	(Priority)
54484594	cpu	dock6t6r	aivy_le	PD	0:00	1	(Resources)

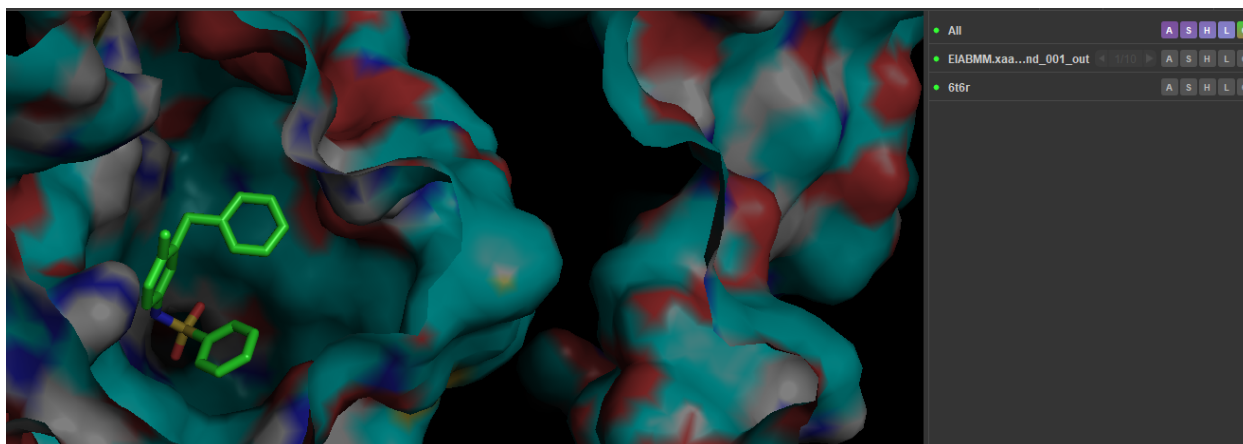
7. **Let it dock:** You will receive an email once it is done or if the time is almost out. Using these specific parameters and settings, it took around 7 hours to finish.
8. **Outputs files:** Once it finishes docking, each protein folder should contain a `.log` and `out.pdbqt` file for each ligand it docked with. Here is an example of what it would look like in one of the protein folders:

```
EIABRN.xaa_ligand_00885_log.log
EIABRN.xaa_ligand_00885_out.pdbqt
EIABRN.xaa_ligand_00886_log.log
EIABRN.xaa_ligand_00886_out.pdbqt
EIABRN.xaa_ligand_00887_log.log
EIABRN.xaa_ligand_00887_out.pdbqt
EIABRN.xaa_ligand_00888_log.log
EIABRN.xaa_ligand_00888_out.pdbqt
EIABRN.xaa_ligand_00889_log.log
EIABRN.xaa_ligand_00889_out.pdbqt
EIABRN.xaa_ligand_00890_log.log
EIABRN.xaa_ligand_00890_out.pdbqt
EIABRN.xaa_ligand_00891_log.log
EIABRN.xaa_ligand_00891_out.pdbqt
EIABRN.xaa_ligand_00892_log.log
EIABRN.xaa_ligand_00892_out.pdbqt
EIABRN.xaa_ligand_00893_log.log
EIABRN.xaa_ligand_00893_out.pdbqt
EIABRN.xaa_ligand_00894_log.log
EIABRN.xaa_ligand_00894_out.pdbqt
```

The *.log* file shows the top binding affinities for the protein/ligand interaction.

```
EIABMM.xaa_ligand_001_log.log
EIAB_6t6r > EIABMM.xaa_ligand_001_log.log
1 #####
2 # If you used AutoDock Vina in your work, please cite: #
3 # #
4 # O. Trott, A. J. Olson, #
5 # AutoDock Vina: improving the speed and accuracy of docking #
6 # with a new scoring function, efficient optimization and #
7 # multithreading, Journal of Computational Chemistry 31 (2010) #
8 # 455-461 #
9 # #
10 # DOI 10.1002/jcc.21334 #
11 # #
12 # Please see http://vina.scripps.edu for more information. #
13 #####
14
15 Output will be ../EIAB_FULL/EIABMM.xaa_ligand_001_out.pdbqt
16 Detected 192 CPUs
17 WARNING: at low exhaustiveness, it may be impossible to utilize all CPUs
18 Reading input ... done.
19 Setting up the scoring function ... done.
20 Analyzing the binding site ... done.
21 Using random seed: 1337474396
22 Performing search ... done.
23 Refining results ... done.
24
25 mode | affinity | dist from best mode
26 | (kcal/mol) | rmsd l.b. | rmsd u.b.
27 +-----+-----+-----+-----+
28 1 -8.2 0.000 0.000
29 2 -7.8 3.210 6.503
30 3 -7.8 1.153 1.514
31 4 -7.7 2.158 5.161
32 5 -7.6 3.355 5.074
33 6 -7.6 1.739 4.652
34 7 -7.6 1.817 4.631
35 8 -7.4 2.077 2.967
36 9 -7.3 1.776 4.666
37 10 -7.1 2.889 5.643
38 Writing output ... done.
39
```

The *out.pdbqt* file is a 3D visual of the protein ligand interaction. Using a molecular visualization software such as [PyMol](#), users can visualize the conformation of each of the 10 binding modes shown in the *.log* file.



9. **Extract binding affinities:** run another python script to extract the binding affinities of one docking job into a csv. There is a script named [LigandIDAffinity.py](#) that is provided on GitHub. This script will create a *.csv* containing the ZINC ID of each ligand and the top

binding affinity. Here is an example of what the file would look like when opened in Excel:

1	Ligand	ZincID	Affinity
2	EIABMM.xaa_ligand_001	ZINC000005516164	-8.2
3	EIABMM.xaa_ligand_002	ZINC000408901016	-7
4	EIABMM.xaa_ligand_003	ZINC000409113807	-7.6
5	EIABMM.xaa_ligand_004	ZINC000409115477	-7.4
6	EIABMM.xaa_ligand_005	ZINC000409216350	-6.8
7	EIABMM.xaa_ligand_006	ZINC000001086666	-8.6
8	EIABMM.xaa_ligand_007	ZINC000103760049	-7.8
9	EIABMM.xaa_ligand_008	ZINC000005568072	-7.7

To find out whether any of these ligands are selective for one protein, the top binding affinities must be extracted for each protein. Therefore, [LigandIDAffinity.py](#) will be run for each protein: 6t6r (ERAP1), 6ea4 (ERAP2), and 5mj6 (IRAP). Once we have all three .csv files, they can be combined into one to compare the binding affinities to each other. Then, they may be filtered out based on the differences in binding affinities. A selective ligand should have a higher binding affinity to one protein compared to the other two proteins.

Conclusion & Data Interpretation

In summary, this tutorial demonstrated how to use the UNITY high-performance computing platform to perform large-scale molecular docking with AutoDock Vina. By distributing docking jobs across multiple computing nodes, users can screen large libraries of ligands against multiple proteins in parallel. This significantly accelerates the molecular docking workflow and allows users to start data interpretation much faster.

The output files from molecular docking consist of both quantitative and structural results. The files show the predicted binding affinity of each ligand to the protein and the 3-D structure of the binding poses. Together, these outputs allow researchers to evaluate the strength of the protein-ligand interaction and the molecular position.

Additionally, after extracting the top binding affinities for each protein as seen in [step 9](#), the results for all three proteins can be combined and compared to find potential selective ligands. Compounds that exhibit stronger predicted binding affinities to one protein compared to others may be promising candidates for drug development. In this case, the more negative the binding affinity, the stronger the interaction. An example of selectivity is shown below:

	Binding Affinity (kcal/mol)		
	6T6R (ERAP1)	6EA4 (ERAP2)	5MJ6 (IRAP)
Ligand 1	-4.0	-3.2	-3.5
Ligand 2	-11.2	-5.0	-4.2

Selective to 6T6R (ERAP1)

Overall, this computational workflow is a great initial screening step for drug discovery. Researchers can narrow down thousands of potential compounds to a smaller list of high-priority candidates that can be focused on in experimental validation steps. By using a high-performance computing platform, researchers can screen many more ligands and increase the chances of identifying an effective and selective modulator.

References

- Agu, P.C., Afiukwa, C.A., Orji, O.U. *et al.* Molecular docking as a tool for the discovery of molecular targets of nutraceuticals in diseases management. *Sci Rep* 13, 13398 (2023). <https://doi.org/10.1038/s41598-023-40160-2>
- Barlow, N., & Thompson, P. E. (2020). IRAP Inhibitors: M1-Aminopeptidase Family Inspiration. *Frontiers in pharmacology*, 11, 585930. <https://doi.org/10.3389/fphar.2020.585930>
- Liddle, John, et al. “Targeting the Regulatory Site of ER Aminopeptidase 1 Leads to the Discovery of a Natural Product Modulator of Antigen Presentation.” *Journal of Medicinal Chemistry*, vol. 63, no. 6, 28 Feb. 2020, pp. 3348–3358, <https://doi.org/10.1021/acs.jmedchem.9b02123>
- Puertas-Martín, S., Banegas-Luna, A. J., Paredes-Ramos, M., Redondo, J. L., Ortigosa, P. M., Brovarets', O. O., & Pérez-Sánchez, H. (2020). Is high performance computing a requirement for novel drug discovery and how will this impact academic efforts? *Expert Opinion on Drug Discovery*, 15(9), 981–985. <https://doi.org/10.1080/17460441.2020.1758664>
- Sui, L., & Guo, H. C. (2021). ERAP1 binds peptide C-termini of different sequences and/or lengths by a common recognition mechanism. *Immunobiology*, 226(4), 152112. <https://doi.org/10.1016/j.imbio.2021.152112>